CPT304 Coursework 1 Report
Taskify Research-Led Software Enhancement

Group XX

May 2026

1 App Selection

Taskify was chosen because it is a task-management app with a simple user journey: a visitor signs up or logs in, then manages tasks on the dashboard. That made it practical to audit the project from the page templates through to the Express routes and tests. The app was also not already polished to the coursework baseline. It had issues in accessibility, backend form handling, persistence, test coverage, and compliance pages. Those gaps made it a good project for small, traceable fixes rather than a broad rewrite.

2 Authentication Form Fields Relied on Placeholder Text

2.1 Detection

The issue was first found while checking `views/signup.ejs`. The sign-up and login inputs had placeholder text, but the user-entered fields did not have their own visible labels or matching `id`/`for` pairs. This was easy to miss because the page looked acceptable before typing. The problem appeared once text was entered: the placeholder disappeared, leaving the filled username field without a visible field name. The browser evidence in Figure 1 shows that exact before and after state.

2.2 Literature

WCAG 2.2 Success Criterion 3.3.2 requires labels or instructions when content needs user input [1]. W3C Technique H44 gives the implementation detail used here: explicit `label` elements can be connected to controls through matching `for` and `id` values, and visible labels also help users understand the field before and after typing [2]. This made the fix semantic rather than only visual.

2.3 Implementation

The original field relied on the placeholder to describe the input:
Before fix: placeholder-only email field.

```
<input type="email" name="SignUpEmail"
  placeholder="Email" required="true" />
```

The updated page keeps the hint text, but adds a real label and a stable association:

After fix: visible label connected to the input.

```
<label class="field-label" for="signup-email">Email</label>
<input id="signup-email" type="email" name="SignUpEmail"
  autocomplete="email" placeholder="Email" required />
```

The same pattern was applied to the username, password, login email, and login password fields. The CSS was also split so the large panel switch labels use `.toggle-label`, while normal input labels use `.field-label`. That avoided turning every form label into a page heading. The final result keeps the field name visible after typing and gives assistive technology a clear label-to-control relationship.

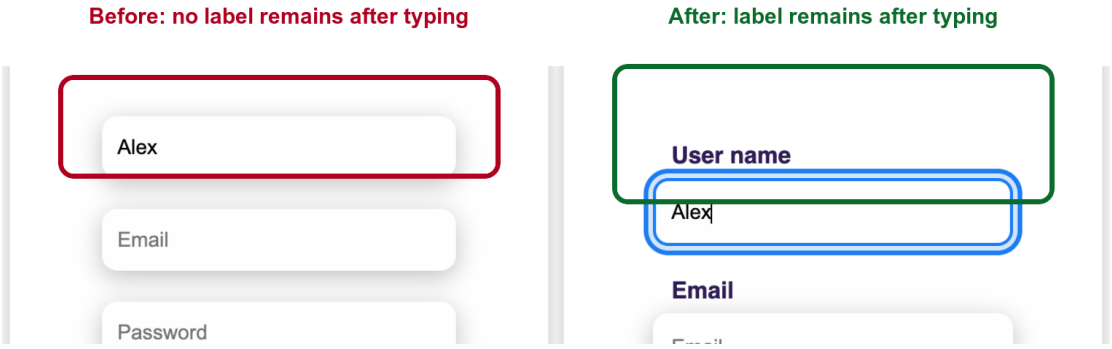

Figure 1: Authentication form label evidence. Before the fix, the username placeholder disappears after typing and no field label remains. After the fix, the visible “User name” label stays above the filled input.

3 Authentication Forms Submitted to Missing Backend Routes

3.1 Detection

The next issue came from following the same forms through to the backend. In `views/signup.ejs`, the sign-up form submitted `POST /signup` and the login form submitted `POST /login`. In the earlier `src/app.js`, Express only defined `GET /`, `GET /signup`, and `GET /dashboard`. A direct curl check confirmed the bug: a valid `POST /signup` returned `HTTP/1.1 404 Not Found`. In practice, a user could fill the form correctly and still be sent to a missing route.

3.2 Literature

Express routing is matched by both HTTP method and path, so `GET /signup` does not handle `POST /signup` [3]. OWASP’s Input Validation Cheat Sheet was used because direct POST requests can bypass browser validation, so required fields and email format still need server-side checks [4]. RFC 9110 was used for the success path: `303 See Other` redirects the user agent to a different resource identified by the `Location` header, which fits a Post/Redirect/Get flow to `/dashboard` [5].

3.3 Implementation

The earlier backend rendered the sign-up page but did not process its submission:
Before fix: sign-up page existed, but no POST handler existed.

```
app.get("/signup", (req, res) => {
    res.status(200).render("signup.ejs");
});
```

The fix added method-specific POST handlers for the existing form actions:
After fix: server-side validation and Post/Redirect/Get.

```
app.post("/signup", (req, res) => {
    const missingFields = collectMissingFields(req.body, [
        "SignUpUsername", "SignUpEmail", "SignUpPassword",
    ]);

    if (missingFields.length > 0 || !isValidEmail(req.body.SignUpEmail)) {
        return res.status(400).json({
            success: false,
            errors: {
                missingFields,
                email: isValidEmail(req.body.SignUpEmail) ? undefined
                    : "Enter a valid email address.",
            },
        });
    }

    return res.redirect(303, "/dashboard");
});
```

The same approach was added for `POST /login`. The app now rejects invalid email input with `400 Bad Request` and redirects valid submissions with `303 See Other`. To make this testable, `src/app.js` exports the Express app and only starts `listen()` when run directly, so `node --test` can run the route checks on a temporary port.

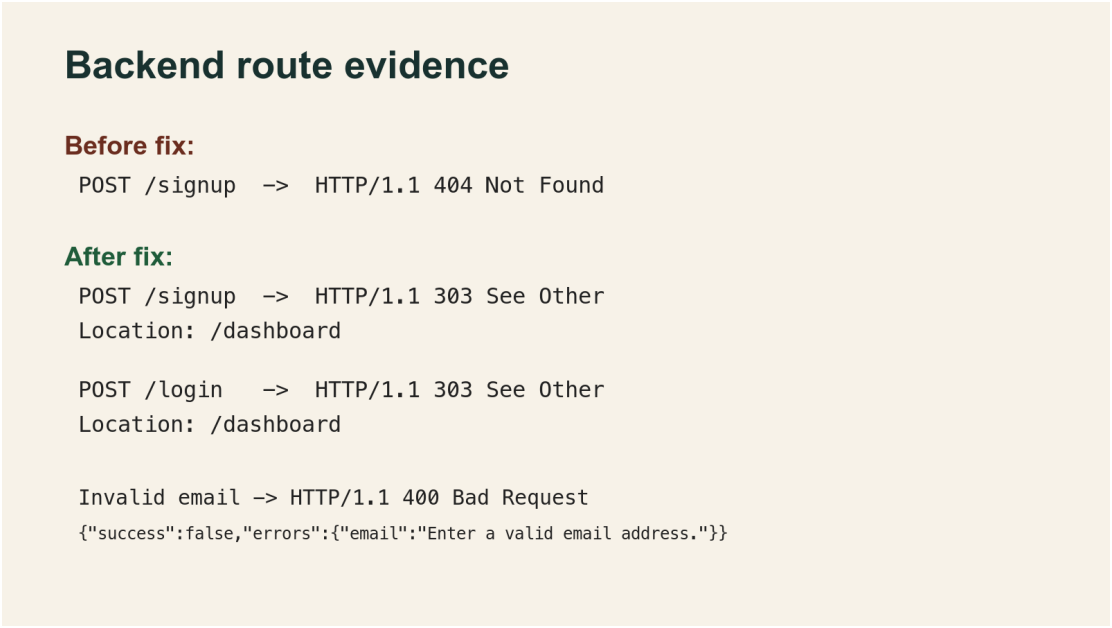

Figure 2: Backend route evidence. Before the fix, a valid `POST /signup` returned 404. After the fix, valid sign-up and login submissions return `303 See Other` to `/dashboard`, while invalid email input is rejected with `400 Bad Request`.

4 Dashboard Tasks Were Lost After Refresh

4.1 Detection

The third issue appeared during a refresh check on the dashboard task board. A new task could be created and shown on the board, but after refreshing the page the board went back to the three sample tasks. The saved item, `Persistence evidence task`, was gone. The cause was in `static/js/dashboard.js`: the initial task state was rebuilt from `seedTasks` every time the script loaded, and the updated task array was not written anywhere durable. Figure 3 shows the full check: the task first exists after saving, then disappears after refresh in the old version, and finally remains after refresh in the fixed version.

4.2 Literature

MDN describes the Web Storage API as a browser feature for storing key-value data on the client, with `localStorage` persisting beyond a single page session [6]. The W3C Web Storage specification defines `getItem` and `setItem` as origin-scoped storage operations, so it was a reasonable fit for a small dashboard task list that did not yet have a database-backed task model [7]. Mendoza et al. also describe HTML5 browser storage as native client-side support for persistent data storage, which supports using browser storage for lightweight state that should survive a refresh [10]. The loader still needed a safe fallback, because browser storage can be empty, cleared, or contain invalid JSON. In those cases, the dashboard should return to the sample tasks instead of failing to render.

4.3 Implementation

The earlier dashboard always rebuilt state from the same seed data:
Before fix: dashboard state was recreated on every load.

```
let state = {
  tasks: seedTasks.map((task) => ({ ...task })),
};

function deleteTask(taskId) {
  state.tasks = state.tasks.filter((task) => task.id !== taskId);
  renderBoard();
  showMessage("Task deleted successfully.");
}
```

The fix moved persistence into a small module and loaded the dashboard from `localStorage` when valid saved data exists:
After fix: state is loaded safely and saved after changes.

```
import { persistTasks, safeLoadTasks } from "./modules/task-store.mjs";

let state = {
  tasks: safeLoadTasks(window.localStorage,
    seedTasks.map((task) => ({ ...task }))).tasks,
};

function persistState() {
  persistTasks(window.localStorage, state.tasks);
}

function deleteTask(taskId) {
  state.tasks = state.tasks.filter((task) => task.id !== taskId);
  persistState();
  renderBoard();
}
```

The new `task-store.mjs` file keeps the storage key in one place and wraps `JSON.parse` in a `try`/`catch`. If saved data is missing, invalid JSON, or not an array, `safeLoadTasks` returns the fallback seed tasks instead of throwing an error. The dashboard now calls `persistState()` after save, delete, and move actions. The page script was also changed to `type="module"` so the dashboard can import the storage helper cleanly.

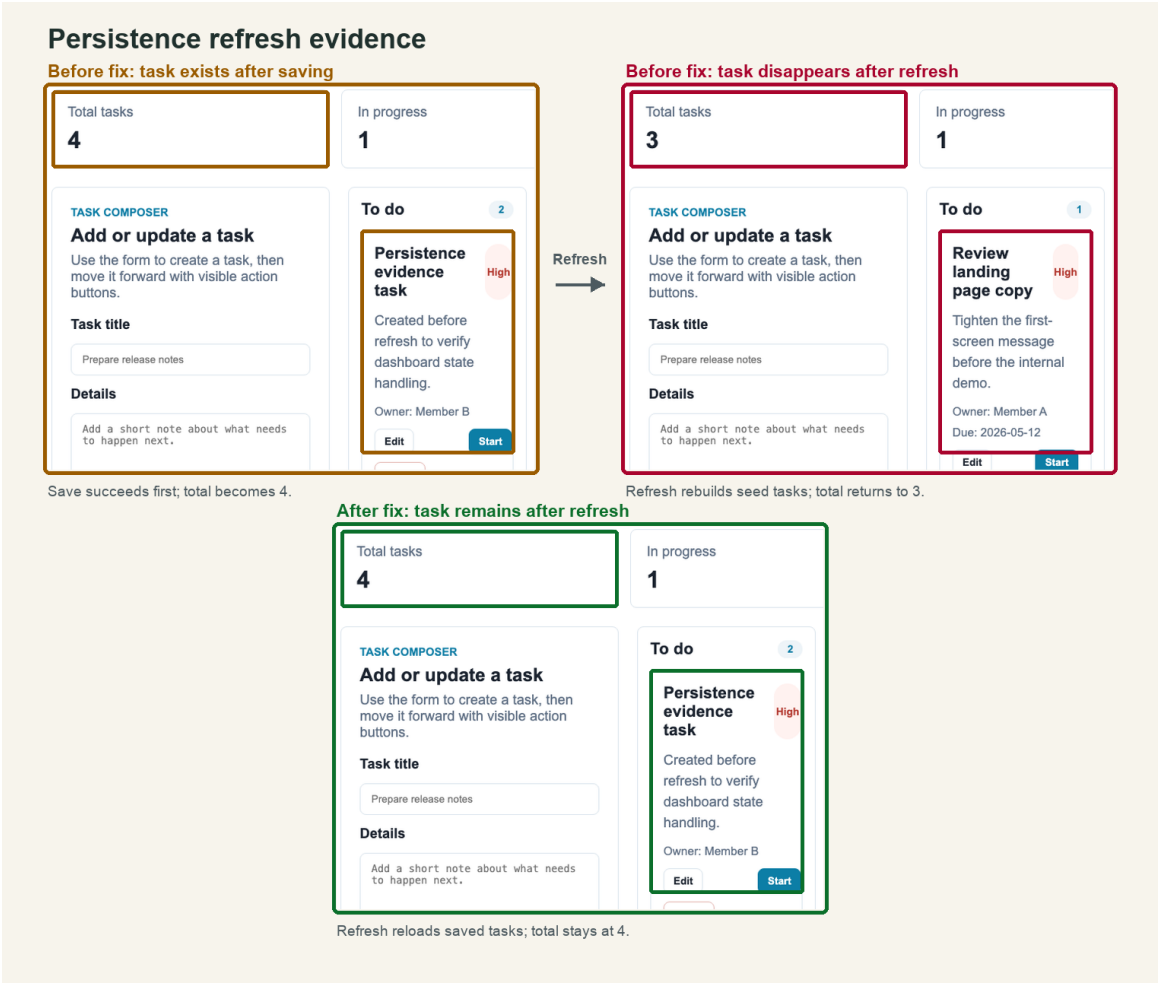

Figure 3: Persistence refresh evidence. The first pre-fix panel shows that the task was created successfully before refresh. The second pre-fix panel shows the defect: refreshing the dashboard rebuilds the board from seed tasks, so the total returns to three and the saved task is missing. After the fix, the same refresh keeps the saved task because the board reloads the `localStorage` task list.

5 Weak User Feedback and Unclear Task Actions

5.1 Detection

This issue was identified by reviewing the earlier dashboard interface and tracing the task workflow from the user perspective. The original dashboard mainly presented static task cards and did not give users a clear, interactive sequence for managing work items. Important actions such as editing a task, moving it forward in the workflow, resetting its status, or deleting it were either unavailable or not clearly expressed as direct controls. As a result, the dashboard looked like a board, but did not behave like a usable task-management tool.

The feedback problem was equally visible during interaction analysis. When a user created, edited, or changed a task, there was little or no immediate confirmation explaining what had happened. This meant that users had to infer success from the layout alone rather than being explicitly informed. In a task-management interface, that

is a usability problem because small state changes happen frequently and should be communicated clearly.

5.2 Literature

This improvement was informed by both an academic usability article and a professional UX article, each of which emphasises the importance of timely feedback and clear interaction flow. Barrington argues that usable systems should communicate system state through appropriate feedback within a reasonable time and should support navigation through clearly marked and recoverable actions [8]. These principles are directly relevant to a task dashboard, where users need to understand both the outcome of an action and the next available workflow step.

A similar position is presented in a professional UX article published on DEV Community, which explains that interfaces should keep users informed about ongoing system changes and should make common actions understandable and reversible [9]. Taken together, these sources provide a clear rationale for introducing visible task controls, explicit workflow transitions, and immediate confirmation messages in the dashboard.

5.3 Implementation

The dashboard was redesigned so that each task card exposes explicit action buttons and each important action produces visible interface feedback. The implementation was mainly completed in `static/js/dashboard.js`, supported by updates to the dashboard template and styling.

Before the improvement, the dashboard section functioned mostly as a visual mockup of task columns. The interface did not provide a complete user-facing workflow with direct task actions and clear system responses.

Before improvement: static dashboard content with limited user action flow.

```
<div class="dashboard-task task-1">
  <div class="dashboard-task-head">
    Lorem ipsum dolor sit amet.
  </div>
  <div class="dashboard-task-description">
    Lorem ipsum dolor sit amet consectetur adipisicing elit.
  </div>
</div>
```

After the improvement, task actions were implemented directly in JavaScript and rendered as explicit controls on each card:

After improvement: explicit task actions and visible feedback.

```
actions.append(
  createActionButton("Edit", () => {
    fillForm(task);
    form.elements.title.focus();
    showMessage("Task loaded for editing.");
  })
);

if (task.status === "todo") {
  actions.append(createActionButton(
```

```
    "Start",
    () => moveTask(task.id, "doing", "Task moved to In progress."),
    "primary"
  ));
}
```

The implementation introduced five concrete changes to the dashboard interaction flow. First, each card now exposes clear task action buttons such as `Edit`, `Start`, `Complete`, `Reset`, and `Delete`. Second, the interface now displays visible status messages after important events, such as saving, updating, deleting, loading a task for editing, or moving it to another workflow stage. Third, the board metrics are updated live so that users can immediately observe the current number of total, active, and completed tasks. Fourth, task transitions are now explicit, making it easier to follow movement from *To do* to *In progress* and then to *Completed*. Finally, the dashboard layout and styling were updated so that these controls appear as part of a coherent task-management workflow rather than as decorative interface elements.

These changes follow the literature closely. The action buttons improve user control by making the available operations visible and predictable. The status messages and live metrics improve visibility of system status by telling the user exactly what has happened after each interaction.

5.4 Evidence

The improved dashboard can now demonstrate several visible feedback states for the report evidence:

- a success message after saving or updating a task;
- an edit feedback message after loading a task into the form;
- a status-transition message after moving a task to *In progress* or *Completed*;
- updated task metrics showing the new board state after interaction.

A suitable final report screenshot should therefore show a task moving between columns together with the visible confirmation message and the updated metric counters. This demonstrates that the revised dashboard communicates interaction outcomes explicitly, rather than requiring the user to infer state changes indirectly from the layout alone.

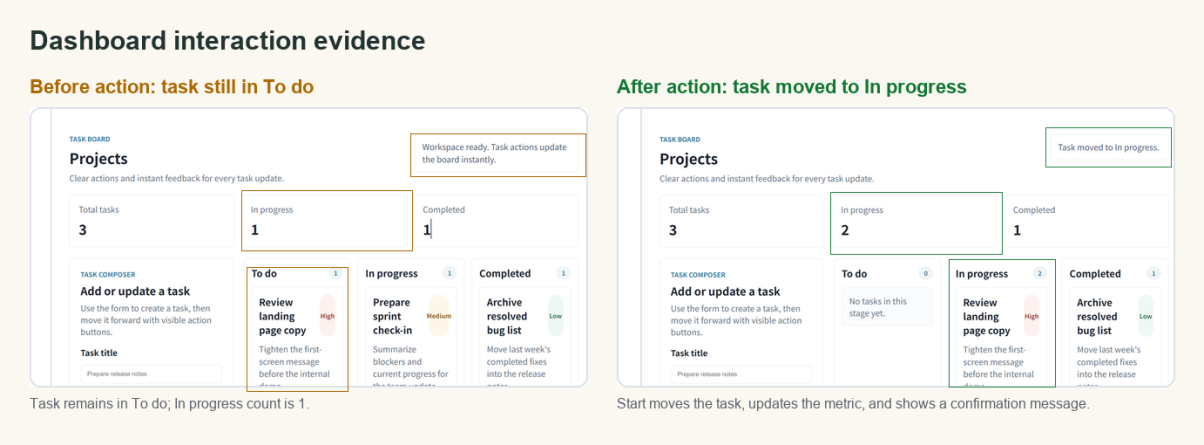

Figure 4: Dashboard interaction evidence. Before the improvement, the selected task was still in the To do column and the interface only showed the default workspace message. After the user selected Start, the task moved to In progress, the system displayed a visible confirmation message, and the dashboard metrics updated immediately.

6 Baseline Standards Evidence

TODO: Add concise evidence and screenshots for the required baseline standards: 7-day Vercel/Render uptime, Codecov/Istanbul coverage of at least 80%, Lighthouse Accessibility score of at least 90, a two-language i18n toggle, a cookie banner, and a privacy policy page. Each screenshot must have a figure number and descriptive caption.

7 Individual Contribution Forms

The completed contribution workbook is included as a separate submission artifact: `individual-contribution.xlsx`. This file is excluded from the report word count and is the place where individual contribution evidence and pull-request links are recorded.

8 References

[1] W3C Web Accessibility Initiative, "Understanding Success Criterion 3.3.2: Labels or Instructions." [Online]. Available: https://www.w3.org/WAI/WCAG22/Understanding/labels-or-instructions.html. Accessed: May 8, 2026.

[2] W3C Web Accessibility Initiative, "H44: Using label elements to associate text labels with form controls." [Online]. Available: https://www.w3.org/WAI/WCAG22/Techniques/html/H44. Accessed: May 8, 2026.

[3] Express, "Routing." [Online]. Available: https://expressjs.com/en/guide/routing.html. Accessed: May 8, 2026.

[4] OWASP Cheat Sheet Series, “Input Validation Cheat Sheet.” [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html. Accessed: May 8, 2026.

[5] R. Fielding, M. Nottingham, and J. Reschke, “HTTP Semantics,” RFC 9110, Jun. 2022. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9110. Accessed: May 8, 2026.

[6] MDN Web Docs, “Web Storage API.” [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API. Accessed: May 8, 2026.

[7] W3C, “Web Storage (Second Edition).” [Online]. Available: https://www.w3.org/TR/webstorage/. Accessed: May 8, 2026.

[8] S. Barrington, “Usability in the Lab: Techniques for Creating Usable Products,” *Journal of Laboratory Automation*, vol. 12, no. 1, 2007. [Online]. Available: https://doi.org/10.1016/j.jala.2006.08.004. Accessed: May 8, 2026.

[9] Lollypop Design, “What are the 10 Principles of Usability Heuristics?,” DEV Community, Jan. 23, 2025. [Online]. Available: https://dev.to/lollypopdesign/what-are-the-10-principles-of-usability-heuristics-1nb1. Accessed: May 8, 2026.

[10] A. Mendoza, A. Kumar, D. Midcap, H. Cho, and C. Varol, “BrowStEx: A tool to aggregate browser storage artifacts for forensic analysis,” *Digital Investigation*, vol. 14, pp. 63–75, 2015. [Online]. Available: https://doi.org/10.1016/j.diin.2015.08.001. Accessed: May 9, 2026.